A FREE CHAPTER FROM

Metrics & Mayhem

Observability That Actually Works

CHAPTER FOUR

Who Owns What When It Breaks

Allan Mann
Cleartext Press

Before you read this

If you are reading this, you are on the Mastering Observability list. That means you have been giving me your attention for a while now. A free chapter is the smallest way I can think of to say thank you for it.

Metrics & Mayhem is out now. Eleven chapters on why observability spend keeps climbing while recovery times keep getting worse, and what to do about it. The whole book rests on one uncomfortable idea: this was never a tooling problem. It is a leadership problem.

The chapter in front of you is Chapter 4, “Who Owns What When It Breaks.” I chose it on purpose. It is the chapter that holds the book together. It takes the most expensive IT failure of the last decade, the CrowdStrike outage, and asks the question almost nobody asked at the time: why did one airline recover in a day and another take five? The answer is not technical. It is a model you can draw on a whiteboard and use on Monday.

Read it on its own terms. If it earns the next chapter from you, the rest of the book is one click away.

Allan Mann

What the rest of the book covers

Eleven chapters. Each one earns a single decision before the next.

1. Why This Is a Board-Level Problem
The MTTR Paradox: more tools, more spend, slower recovery. Why this belongs on the board's risk register.

2. What Observability Actually Means for a CTO
Monitoring watches for the problems you predicted. Observability answers the question you did not.

3. What It's Really Costing You
The licence fee is the visible cost. The real bill is everything below the waterline.

4. Who Owns What When It Breaks
The chapter you are about to read: the three-layer ownership model, and the CrowdStrike test.

5. The People Problem Nobody's Solving
Training is not capability. Why the tools you bought are still not being used.

6. Why Your Best Engineers Aren't Telling You the Truth
Psychological safety, and the cost of the signal that never gets spoken aloud.

7. The Machines Are Getting Better. Are Your Teams?
What AI in operations actually changes for your people, and what it does not.

8. The CTO Nobody Trained You to Be
The three shifts that outpaced the role, and the three numbers your board actually needs.

9. What the Organisations That Get This Right Actually Did
GitHub, Fastly and Atlassian: three public failures, three structural recoveries.

10. How to Build This Without Burning It Down
The phased transformation: one outcome, one owner, one quarter.

11. What to Do Monday Morning
Three conversations this week, a 90-day plan, and exactly where to start.

CHAPTER FOUR

Who Owns What When It Breaks

CTO question: "Why does nobody know who's responsible during an incident?"

P1 incident. Seventeen people on the bridge. The checkout service is returning 503 errors on approximately 30% of requests. The platform team runs their checks: infrastructure healthy, all pods running, no resource exhaustion. The application team checks their service: response times nominal from their perspective, no error spikes in their logs. The payments team checks their provider: gateway API returning 200s within SLA. Three teams, three green dashboards. And the customer is staring at a spinning wheel where the "Complete Purchase" button should be.

Seventeen people. One bridge call. Three teams pointing at each other's dashboards.

The incident commander asks the question that should be simple: "Who owns the checkout experience end to end?" Silence. Because the answer is: nobody. The platform team owns infrastructure. The application team owns the service. The payments team owns the provider integration. But the outcome, the thing the customer actually experiences, whether they can complete a transaction, that lives in the gaps between those three teams, and nobody has been assigned to own the gaps.

This is not a tooling problem. Every team's monitoring was working correctly. The infrastructure was healthy. The application was responding. The payment gateway was up. The failure was in a load balancer configuration that was routing a subset of traffic to a backend pool that had been decommissioned during a migration three weeks earlier. It fell between the application team's monitoring (which only measured their own service health) and the platform team's monitoring (which checked the load balancer's availability but not its routing accuracy).

The 503s were real. The customer impact was real. And every dashboard said green because every dashboard was answering a different question than the one the customer was asking: can I pay?

I have been in that room. I have been the incident commander asking that question and hearing silence. I have watched good engineers spend forty minutes debugging something that was not broken while the actual problem sat in a configuration that nobody's monitoring covered because nobody owned the outcome it affected.

The Three-Layer Model

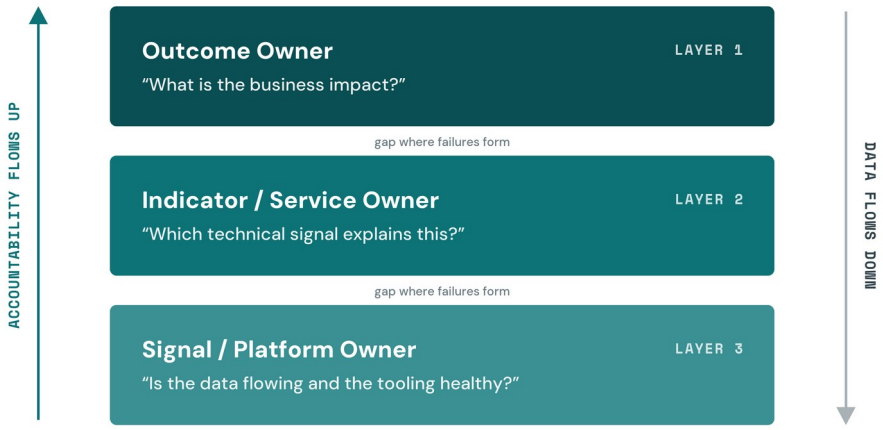

Figure 4.1 Each layer answers a different question. The gap between layers is where most observability failures occur.

The operating model for observability needs three layers of ownership. Not one. Not "the platform team handles it." Three distinct layers, each with a named team and clear accountability.

Layer 1: Outcome owners. The team accountable for a business result. In the payments context, this means the team that can answer the question: "What is our payment completion rate right now, and why did it change?" This is not the same as owning the payment service. It is owning the measurable business outcome that the payment service contributes to. The outcome owner does not need to write code. They need the authority to convene the teams whose systems affect the outcome, the visibility to see when the outcome degrades, and the accountability to explain what happened.

Layer 2: Indicator owners. The teams responsible for the technical signals that explain the outcome. For payment completion rate, these signals include checkout page load time (p99), payment gateway latency, authentication service availability, cart timeout rates, and load balancer routing accuracy (which, in our opening scenario, was the missing signal). Each signal has a team that maintains the instrumentation, sets the thresholds, and responds when their signal degrades. The indicator

owner's job is to know what "normal" looks like for their signal and to raise their hand when it doesn't.

Layer 3: Platform owners. The team that maintains the observability tooling, the telemetry pipelines, the data retention policies, and the instrumentation standards. This is the layer most organisations already have, and it is the layer they mistake for the whole model. The platform team ensures the infrastructure works. They do not (and should not) own business outcomes.

Most organisations I've worked with have Layer 3. Some have started to build Layer 2, usually in the form of SRE teams that own specific service-level indicators. Very few have Layer 1. And Layer 1 is the one that actually matters during an incident, because it is the only layer that can answer the question the business is asking: what happened to the customer?

Reality check: if nobody can say "this outcome is mine" during a P1, your operating model is a diagram, not a system.

Why Ownership Gaps Form

Ownership gaps do not form because people are careless. They form because the organisational structure was designed around technical components, not business outcomes.

The typical enterprise engineering organisation is structured by system. There is a team that owns the database layer, a team that owns the application services, a team that owns the network, and a team that owns the front end. This makes sense for building and maintaining those components. It does not make sense for understanding what happens when a customer's request traverses all of those components in sequence and any one of them can cause the request to fail.

The payment completion path in a modern e-commerce platform might cross six teams' infrastructure in a single request: the CDN, the front-end service, the API gateway, the checkout service, the payment processor integration, and the database that records the transaction. Each team monitors their piece. Nobody monitors the path.

This is not a coordination failure. It is a design failure. The organisation was never designed to have someone own the path. It was designed to

have people own the pieces. And when the path breaks in a way that no individual piece explains, the result is a bridge call with seventeen people, each holding a green dashboard, while the customer cannot pay.

The Chronosphere survey data from Chapter 3 reinforces this: 43% of engineers say their alerts lack enough context to begin triage.[1] That is not a tooling limitation. It is the direct consequence of alerts that are tied to technical components rather than business outcomes. An alert that says "database latency exceeded 100ms" tells you something is slow. It does not tell you whether customers can still complete transactions. That second question requires someone to have connected the technical signal to the business outcome, and that connection is Layer 1 work that most organisations have never done.

CrowdStrike: The $5.4 Billion Ownership Test

On 19 July 2024, CrowdStrike pushed a faulty sensor configuration update to its Falcon Endpoint Detection and Response software. A logic error in Channel File 291 caused Windows systems running Falcon to crash with Blue Screen of Death. Approximately 8.5 million Windows devices went down globally. Airlines, banks, hospitals, emergency services, retailers, broadcasters. The estimated damage to Fortune 500 companies alone exceeded $5.4 billion.[2]

The same update hit every organisation running CrowdStrike on Windows. The same Blue Screen. The same requirement for manual intervention: boot into Safe Mode, navigate to the CrowdStrike directory, delete the faulty file, restart. No remote fix was available for most machines.

What happened next was the most expensive ownership test in the history of IT.

American Airlines recovered by the evening of 19 July. Minimal cancellations the following day. United Airlines took approximately three days, cancelling around 1,400 flights. Delta Air Lines did not resume normal operations until 25 July. Five full days. Over 7,000 flights cancelled. 1.3 million passengers affected. The airline reported $380 million in lost revenue and $170 million in additional costs. Total claimed losses: $500 million.[3]

Same incident. Same root cause. Radically different outcomes. The difference was not technical skill. It was ownership of the end-to-end operational readiness question that nobody at Delta had been assigned to ask: what happens to our ability to operate flights if this entire software layer fails simultaneously?

What Delta Missed

Delta's crew-tracking system was the critical failure point. This is the system that pairs pilots and flight attendants with flights, ensuring every departure has a legally compliant crew in position. Without it, planes can sit on the tarmac fully fuelled with passengers boarding, and the airline still cannot fly.

Approximately 60% of Delta's mission-critical applications ran on Microsoft Windows with CrowdStrike. That included the redundant backup systems. When the primary systems crashed, the backups crashed too, because they depended on the same software layer.[4] The redundancy was architectural theatre: two copies of the same vulnerability.

Delta had roughly 40,000 servers that required manual reboot. Each one needed physical or console-level access to boot into Safe Mode and delete the faulty file. The crew-tracking system, being deeply complex and interdependent with other applications, required additional synchronisation time even after individual servers were restored.[5]

Nobody at Delta owned the question: "What is our dependency exposure to a single-vendor failure across our operational stack?" The infrastructure team owned servers. The security team owned the CrowdStrike relationship. The operations team owned crew scheduling. But the intersection of those three domains, the place where a security vendor's faulty update could ground an entire airline for five days, sat in a gap between ownership boundaries.

The U.S. Department of Transportation classified the disruption as a "controllable" event, placing responsibility on Delta, not CrowdStrike.[6] In regulatory language, that is an ownership determination. The regulator decided the airline was responsible for its own operational resilience, regardless of which vendor caused the initial failure.

The vendor consolidation pitch sounds compelling right up until you realise the consolidation project itself will take 18 months and cost more than the overlap. But Delta's problem was not consolidation. It was concentration without resilience. They had consolidated onto Windows and CrowdStrike so thoroughly that a single faulty update took out both primary and backup systems. Consolidation without outcome ownership is just a tidier way to create a single point of failure.

What CrowdStrike Got Right

The contrast between how CrowdStrike handled their failure and how Delta handled the consequences is instructive for any CTO thinking about what ownership looks like in practice.

CrowdStrike's CEO George Kurtz posted a public acknowledgement at 5:45 AM ET on 19 July, within hours of the incident. Not a vague "we're investigating" statement. A direct acknowledgement that the root cause had been identified and a fix was available. A preliminary Post-Incident Review was published within five days. A twelve-page Root Cause Analysis followed on 6 August with full technical detail, including the specific code-level explanation of how the logic error occurred.[7]

In September, CrowdStrike's SVP of Counter Adversary Operations testified before the U.S. House Subcommittee on Cybersecurity. He apologised. He detailed specific process changes: updates now treated as code releases with internal testing and phased rollout, a new "concentric rings" deployment model, and customer control over update timing. Two independent third-party security vendors were commissioned to review the Falcon sensor code.

At DEF CON's annual Pwnie Awards, CrowdStrike won "Most Epic Fail." Their president, Michael Sentonas, accepted the award in person.[8]

That is ownership. Not just the apology. The public accountability, the detailed technical transparency, the willingness to stand in a room full of security professionals and accept a trophy for the worst mistake in your company's history. CrowdStrike did not deflect. They did not litigate the narrative. They owned the failure, explained it, and changed the system that produced it.

Compare that with the lawsuit trajectory. Delta hired prominent litigator David Boies to pursue damages. CrowdStrike countersued, alleging

Delta's prolonged disruption was primarily the result of Delta's own operational decisions. Both sides may have legitimate claims. But the ownership question the CTO reading this book should be asking is not "who was legally liable?" It is: "If this happened to my organisation, would we recover in one day like American Airlines, or in five days like Delta? And whose job is it to make sure we've answered that question before the incident happens?"

The Bridge Call Problem

I want to name something that most incident management discussions avoid: the bridge call itself is often the symptom, not the solution.

In my experience across multiple organisations, incidents with more than ten people on the bridge take longer to resolve than incidents with five or fewer. This is a field observation, not a controlled study, and I want to scope it honestly. But the pattern is consistent enough to describe.

When more than ten people join a bridge, the incident commander spends their time on coordination rather than diagnosis. Each new participant needs context. Each context-setting explanation takes time. Side conversations start. Someone asks a question that was answered ten minutes ago. The incident commander becomes a facilitator managing a meeting rather than a leader driving resolution.

The organisations I've seen resolve incidents fastest have a deliberate small-team model. The incident commander, one representative from each relevant team (defined by the outcome affected, not by who might know something), and a communications lead if the incident is customer-facing. Everyone else gets status updates asynchronously. The bridge is not a town hall. It is a working session with clear roles.

This connects directly to the ownership model. If you have outcome ownership defined in advance, you know which teams to pull onto the bridge before the incident starts. The outcome owner for payment completion already knows that the checkout service team, the payments integration team, and the platform team are the relevant parties. You don't need to page seventeen people and sort out who's relevant in real time.

Where This Breaks

Outcome ownership requires organisational authority to match. This is the part that most operating model discussions skip, and it is the part that makes this hard.

If the payments outcome owner is a senior product manager who can define SLOs and convene teams, but who cannot prioritise engineering work or influence infrastructure decisions, the model collapses into politics. The outcome owner identifies the problem ("our payment completion SLO is degrading because of load balancer routing accuracy"). The platform team responds ("we have a backlog of 40 items and routing accuracy monitoring is not in the current sprint"). The outcome owner escalates. The platform team's engineering manager pushes back. The CTO gets pulled into a prioritisation discussion that should have been resolved at the operating model level.

This happens because outcome ownership was layered onto the existing structure without adjusting the authority model. The title was created. The authority was not.

For the three-layer model to work, the outcome owner needs either direct authority over the teams that affect their outcome, or a governance mechanism that ensures their priorities are weighted appropriately in those teams' planning cycles. This is a leadership design problem. It requires the CTO to decide, explicitly, that certain business outcomes carry prioritisation weight in engineering planning. That is a harder conversation than creating a new role on the org chart.

I have seen organisations try to solve this with a RACI matrix. Responsible, Accountable, Consulted, Informed. The matrix gets drawn. It gets filed. Nobody references it during an incident because the incident moves faster than the matrix anticipated. Ownership needs to be simpler than a matrix. It needs to be a name. One person. Who owns this outcome? If the answer requires looking at a spreadsheet, you've already lost the race against the incident.

The Blameless Trap

One more ownership pattern that needs naming. The industry has largely adopted blameless post-incident reviews, and rightly so. Blame cultures

suppress information sharing, discourage transparency, and make incidents worse because people hide what they know.

But blameless does not mean ownerless.

I've sat in post-incident reviews where the word "we" was used so carefully that nobody was accountable for anything. "We missed the alert." "We should have caught the configuration change." "We need to improve our process." Who is "we"? Which team missed the alert? Which person should have reviewed the configuration? Which process, specifically, needs to change, and who is going to change it?

Blameless post-incident reviews exist to create psychological safety so that people share what happened honestly. They do not exist to distribute accountability so broadly that nobody holds it. The outcome owner's role extends past the incident itself. They are accountable for ensuring that the corrective actions identified in the review are actually completed, not just recorded in a Jira ticket that ages into the backlog.

CrowdStrike's response is the model here. They were blameless in the sense that they did not fire someone publicly or scapegoat an individual engineer. But they were not ownerless. The CEO took personal accountability. The SVP testified before Congress by name. The process changes were specific, documented, and externally auditable. That is the balance: psychological safety for individuals, named accountability for outcomes.

Pick your most critical business outcome. If your organisation processes payments, start there. Name one person who owns that outcome end to end. Not the team. The person. If you can't name them, that's your first problem to solve. If you can name them but they don't have the authority to convene the teams whose systems affect their outcome, that's your second problem. And if you're not sure whether your backup systems share the same vulnerability as your primary systems, that's the Delta question, and you need to answer it before the next CrowdStrike-scale event answers it for you. Chapter 5 addresses the third dimension: even with clear ownership, the people in those roles need the skills to act on what the observability stack tells them.

Notes

[1] Chronosphere, "State of Cloud-Native Observability," 2024. 43% of engineers report alerts lack enough context to begin triage.

[2] Parametrix analysis, July 2024. Estimated $5.4 billion impact on Fortune 500 companies, excluding Microsoft. Total global damage estimated at $10 billion or more.

[3] Delta Air Lines SEC filing, August 2024. $380M revenue impact, $170M in additional costs. Delta CEO Ed Bastian cited total losses of $500M in CNBC interview, 31 July 2024. Flight cancellation count (7,000+) and passenger impact (1.3M) from Delta's public statements and DOT investigation.

[4] David Boies letter on behalf of Delta Air Lines, August 2024: "Approximately 60% of Delta's mission-critical applications and their associated data, including Delta's redundant backup systems, depend on the Microsoft Windows operating system and CrowdStrike."

[5] Delta News Hub, "Delta people working 24/7 to restore operation," 22 July 2024. Crew-tracking system described as "deeply complex and requiring the most time and manual support to synchronise."

[6] U.S. Department of Transportation investigation into Delta Air Lines' response to the July 19, 2024 incident. The DOT classified delays and cancellations as a "controllable" event.

[7] CrowdStrike Root Cause Analysis, 6 August 2024 (12-page report). Preliminary Post-Incident Review published 24 July 2024. Congressional testimony by SVP Adam Meyers, 24 September 2024, before the House Subcommittee on Cybersecurity and Infrastructure Protection.

[8] CrowdStrike President Michael Sentonas accepted the 2024 Pwnie Award for "Most Epic Fail" in person at DEF CON.

Where this goes next

That was one chapter. The book has ten more, and they connect.

Chapter 4 gave you the ownership model. Chapter 5 asks why the people in those roles still cannot use the tools you bought them. Chapter 6 asks why your best engineers are not telling you the truth. Chapter 9 shows three organisations that lived through public, catastrophic failures and came out stronger, and exactly what they changed. Chapter 11 turns the whole book into three conversations you can have this week.

Metrics & Mayhem is out now on Amazon in Kindle, paperback and hardback.

GET THE BOOK

One ask. If the book is useful to you, an honest review on Amazon is the single most helpful thing you can do for it. It takes two minutes, and it decides whether the next reader ever finds it.

And if you want more between now and your next read: the *Metrics & Mayhem* podcast is on Spotify and Apple Podcasts, and the *Mastering Observability* newsletter is already in your inbox.

Thank you for reading.
Allan Mann